

The SPO portfolio tree: model v1

What it is, how it is trained, and what the planted-threshold experiments say

Roman Beauvallet - Edo Theory

24 September 2026

Documents: `model_and_training_pipeline.tex` (the model), `experiments_planted_threshold.tex` and `experiments_regime_volatility.tex` (the experiments), `model_history.md` (the dated history). Main Code: `code/spo_tree.py`.

The problem, and the bet

The problem. Allocate between a few assets from a few features, every h days, with fees, under noise and with 15 years of history: a *decision* problem, not a forecasting one.

The bet (`motivations.md`). A small tree that

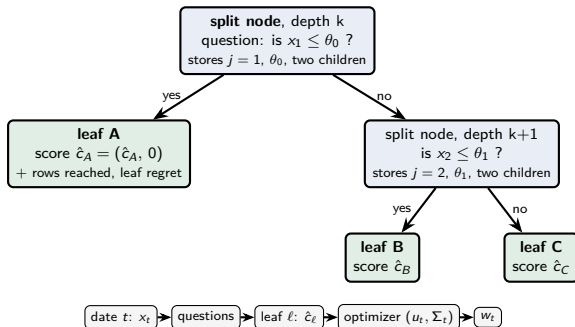
- learns *regimes* rather than a return manifold,
- is trained on the *decision regret* through the optimizer it feeds,
- is judged on its own *traded path*, fees included,

will be more stable, more implementable and interpretable than a larger predictor like a big DNN.

What exists today

- the tree and its exact optimizer, tested (64 unit tests);
- a synthetic market with a *planted* answer, so that recovery can be measured;
- about 15,000 fitted trees over nine experiments (E1–E9);
- a frozen configuration, **model v1**, validated on 100 datasets.

What the tree is, and what its nodes store



How a date is used: answer the questions from the root down, read the leaf's score, let the optimizer turn it into weights from that date's holdings.

A **tree** is a chain of yes/no questions on the features, "is $x_j \leq \theta$?", ending in leaves. The root is the first question; a deeper split node is a question asked only of the rows that reached it; a leaf asks nothing.

- A **split node** (the root included) stores a feature index j , a threshold θ and its two children: yes goes left, no goes right. No score.
- A **leaf** stores one score vector $\hat{c}_\ell$ (asset score, cash at 0), fed to the optimizer for every date that reaches it, plus the number of training rows it received and their leaf regret.
- **Holdings, weights and returns belong to dates**, not to nodes: the tree stores no path.
- Depth at most 3 (v1): at most 8 leaves, hence at most 8 scores. Growth turns one leaf into a split node with two new leaves, we add one leaf at each growth step; pruning does the reverse.

Notation

Notation

- r_t the realised h -day returns of the assets at decision t ; cash has zero return.
- $w_t \in \mathbb{R}^n$: the weights to be decided, including cash; $w \in \mathcal{W}$ (long-only, fully invested, cash included).
- $u \in \mathbb{R}^n$: the weights held *before* the decision, i.e. the previous portfolio after the drift of the period, $u_{t+1} = w_t \circ (1 + r_t)/(1 + r_t^\top w_t)$; $u_1 = \mathbf{1}/n$.
- $c \in \mathbb{R}^n$: the tree's scores, one per asset; cash has score 0.
- Σ : covariance of the h -day returns, from the past only: $h \times$ the covariance of the last 180 daily returns, plus εI ($\varepsilon = 10^{-6}$); for the asset, $\sigma_h^2 = h \hat{\sigma}^2$.
- $f \in \mathbb{R}_{\geq 0}^n$: fee per unit of traded notional (3 bp on the asset, 0 on cash)
- $\lambda > 0$: risk aversion (1 in v1)
- $\bar{w}$: cap per asset (=1).

The decision problem: notation and the exact solver

The problem, one decision every h days over n assets (here a coin and cash, $n = 2$):

$$w^*(c; u, \Sigma) = \arg \max_{w \in \mathcal{W}} \left(c^\top w - f^\top |w - u| - \lambda w^\top \Sigma w \right), \quad \mathcal{W} = \{w \in \mathbb{R}^n : 0 \leq w_i \leq \bar{w}, \sum_i w_i = 1\}$$

Strictly concave ($\lambda > 0$, $\Sigma \succ 0$): one maximizer. Adding a constant to every score changes nothing (it only shifts the objective by a constant), since $\mathbf{1}^\top w = 1$: only score differences matter $\implies n - 1$ degrees of freedom, feature is useful only if it shares relative returns information.

Exact solution by enumeration. Lagrangian, with ν the multiplier of the budget:

$L(w, \nu) = c^\top w - f^\top |w - u| - \lambda w^\top \Sigma w - \nu(\mathbf{1}^\top w - 1)$. At the maximum each weight w_i is in one of five states: at 0; at $\bar{w}$; at u_i (no trade in i : the fee kinks the objective there, so a whole range of scores leaves w_i in place); strictly above u_i (buying); strictly below u_i (selling). In the last two the weight is *free*: no bound and no kink pins it, so $\partial L / \partial w_i = 0$ with the fee's sign known,

$$c_i \mp f_i - 2\lambda(\Sigma w)_i = \nu \quad \text{for every free asset } i, \quad \partial L / \partial \nu = 0 \iff \sum_i w_i = 1,$$

so at the optimum all free assets share the same marginal net gain ν . Fixing the state of every asset (5^n cases) turns these equations into a linear system for the free weights and ν . The solver takes the best objective among them: the optimum is found exactly, for all rows at once (`PortfolioOptimizer.solve`).

Two assets: the exact solution in one line

Asset weight w , cash $1 - w$ (cash: zero return, zero fee, variance ≈ 0). With c the asset's score and u its weight before the decision, the problem is one-dimensional:

$$\max_{0 \leq w \leq 1} g(w) = c w - f |w - u| - \lambda \sigma_h^2 w^2, \quad \kappa := 2\lambda \sigma_h^2.$$

Away from the kink $g'(w) = c \mp f - \kappa w$ (minus when buying, $w > u$; plus when selling, $w < u$); at the kink, u is optimal when 0 lies between the two one-sided derivatives. Hence

$$w^* = \begin{cases} \min \left\{ \frac{c-f}{\kappa}, 1 \right\} & \text{if } c - \kappa u > f \quad (\text{buy}) \\ \max \left\{ \frac{c+f}{\kappa}, 0 \right\} & \text{if } c - \kappa u < -f \quad (\text{sell}) \\ u & \text{if } |c - \kappa u| \leq f \quad (\text{no trade}). \end{cases}$$

Reading. The fee-free target is c/κ : the position is linear in the score with slope $1/\kappa$, so all positions between 0 and 1 correspond to a score window of width κ (from $c = f$ to $c = \kappa + f$ when buying from cash). The fee shifts the target by f/κ towards the current holding and opens a no-trade band, $|c - \kappa u| \leq f$ in score units, f/κ in weight units.

At v1's values ($\lambda = 1$, $\sigma = 1\%$ a day, weekly, $f = 3$ bp): $\sigma_h^2 = 5 \times 10^{-4}$ and $\kappa = 10^{-3}$, so the position goes from 0 to 1 as the score goes from 0.03% to 0.13% a week. The tree's leaf scores are ± 0.3 to 0.5% a week: **positions are 0 or 1**, and the no-trade band (0.3 in weight units) hardly ever binds. With $\lambda = 10$, $\kappa = 1\%$ and the same scores give interior positions, the gradual profiles of Section 8. v1 keeps $\lambda = 1$: at this stage the tree is a regime classifier, not a position sizer.

The loss: decision regret, not forecast error

With the realised returns r_t of decision t and the holdings u_t the tree really had,

$$U_t(w) = r_t^\top w - f^\top |w - u_t| - \lambda w^\top \Sigma_t w,$$
$$\text{Reg}_t(c) = U_t(w^*(r_t; u_t, \Sigma_t)) - U_t(w^*(c; u_t, \Sigma_t)) \geq 0.$$

The regret is the utility lost by deciding on the scores c instead of on the returns that happened, from the same holdings: the SPO loss of Elmachetoub & Grigas with the fees and the risk term inside.

What the leaf learns. The score vector of a leaf minimises the regret summed over its rows,

$$\hat{c}_\ell = \arg \min_{c \in [-b, b]^n} \sum_{t \in \ell} \text{Reg}_t(c), \quad b = 10\%,$$

by a bounded coordinate search (3 passes of 7 points per coordinate). The scores are not forecasts: they are whatever makes the optimizer take the right decision on the leaf's rows.

Three facts that shape everything.

- Only differences of scores matter: $w^*(c + \alpha \mathbf{1}) = w^*(c)$; cash has score 0.
- The regret is measured on the holdings of the current tree, so fees are paid on real trades, not on rebalancing from scratch.
- The regret uses the *same* Σ_t as the optimizer: a risk that the covariance estimate does not see cannot be learnt (slide 20).

The tree, the path, and the two gates

Tree. Internal nodes split on one feature, $x_j \leq \theta$; a leaf holds one score vector $\hat{c}_\ell$. A frozen tree is a map $x \mapsto \hat{c}(x)$, piecewise constant.

Path. Scores are traded in time order from the initial holdings,

$$w_t = w^*(\hat{c}(x_t); u_t, \Sigma_t),$$

$$\pi_t = r_t^\top w_t - f^\top |w_t - u_t|, \quad u_{t+1} = \frac{w_t \circ (1 + r_t)}{1 + r_t^\top w_t},$$

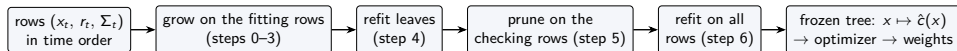
and the path's Sharpe ratio is $S = \text{mean}(\pi)/\text{std}(\pi)$ per decision (0 for an all-cash path). Fees are paid on real trades from drifted holdings.

Growth: two in-sample gates. A candidate split is ranked by its regret reduction $\Delta = L_\ell(\hat{c}_\ell) - L_{\text{left}}(\hat{c}_{\text{left}}) - L_{\text{right}}(\hat{c}_{\text{right}})$ and accepted only if the replayed Sharpe ratio of the whole tree rises.

Pruning: one out-of-sample judge. The last 25% of the training decisions never enter the growth. A split stays only if, there, it lowers the regret and raises the Sharpe ratio: a paired test on the checking rows, with a hurdle in standard errors (v1: 2 s.e., below the root only).

Then every leaf is refitted on all training rows.

Training, step by step



0. **Root.** One score vector on all fitting rows from equal holdings; replay it: holdings u_t , hindsight utilities, Sharpe S .
1. **Every leaf** of depth $< D$: refit it on the current holdings (yardstick); build the candidate thresholds (v1: deciles by rank among the admissible thresholds, then the neighbourhood of the best decile); screen each with a cheap regret $Q(\theta)$ (children scored by their clipped mean return); fit the children of the 3 best; keep a split if $\Delta/N_\ell > 10^{-6}$.
2. **Choice.** Sort the kept splits of all leaves by Δ ; accept the first whose replayed Sharpe ratio beats S ; none passes $\Rightarrow$ growth stops.
3. Replay the new tree: new holdings, utilities, S ; back to step 1 (at most depth $D = 3$, at least 20 decisions and 10% of the parent per leaf).
4. **Refit** every leaf on the final holdings; kept if the Sharpe ratio does not fall.
5. **Prune** bottom-up on the checking rows: gain $g_t = \text{Reg}_t(\text{without}) - \text{Reg}_t(\text{with})$; the split stays iff $\bar{g} > 10^{-6}$, $\bar{g} > k_p \text{se}(\bar{g})$ and the checking Sharpe rises ($k_p = 2$ below the root, 0 at the root).
6. **Refit on all training rows**, rules fixed. The tree is frozen.

Model v1: the frozen configuration (24 September 2026)

Choice	v1	Why (section of the experiments report)
risk aversion λ	1	positions 0/1; the tree is a regime classifier, sizing is not the object yet
fee	3 bp on the asset, 0 on cash	market-sized; 10 bp starves the fast rates
decision rate	weekly ($h = 5$); every 2 days as alternative	daily to weekly equivalent, monthly loses
depth	at most 3	depth alone changes nothing if not reached
leaf minimum	20 decisions and 10% of the node	tail leaves at 0.7% of the data
candidate thresholds	deciles by rank + local refinement (1 s.e.)	same Sharpe as the full screen, steadier thresholds
soft splits	off ($\kappa = 0$)	never raise the Sharpe at $\lambda = 1$
growth gates	regret reduction, then the replayed Sharpe must rise	kept as designed; to be revisited in v2
pruning	last 25% of the training decisions; hurdle 2 s.e. below the root, none at the root	the out-of-sample judge ; the hurdle removes spurious leaves at no cost
covariance	180-day window, $\Sigma_t = h \hat{\sigma}_t^2$	a 20-day window is noisier than it is timely (regime report)
conventions	cash column; all-cash Sharpe = 0; $ \hat{c} \leq 10\%$	(7, model document)

In code: `max_depth=3, min_samples_leaf=20, min_leaf_fraction=0.10, threshold_quantiles=deciles, refine_thresholds=True, refine_standard_errors=1, prune_standard_errors=2, prune_hurdle_from_depth=1, validation_fraction=0.25, fee_rate=(0.0003,0), risk_aversion=1`: the setting V1 of the experiments script. v1 is a *configuration* of pipeline P4; every option it switches on is off by default, so the base of the experiments is P4 with everything off.

The test-bed: a market with a planted answer

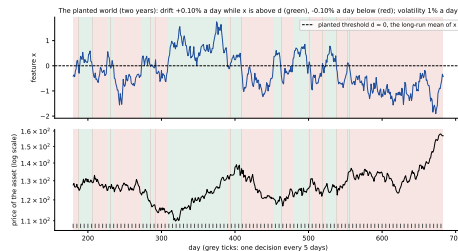
One daily feature x (Ornstein–Uhlenbeck) decides the regime of one asset:

$$dx_t = k(m - x_t)dt + s dW_t$$

$$\frac{ds_t}{s_t} = \mu_t dt + \sigma_t dW_t, \quad \mu_t = \pm \mu \text{ as } x_t \gtrless d,$$

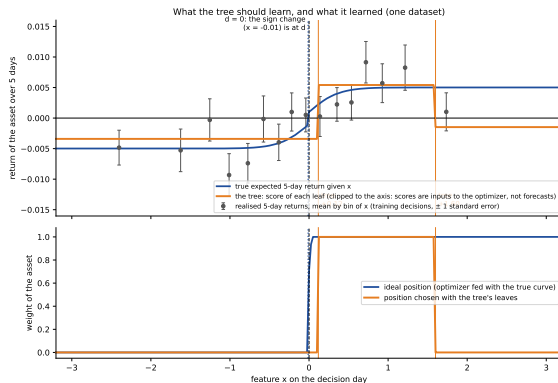
with $\mu = e \sigma$, e the *edge* (daily drift over volatility).

σ	1% a day (16% a year)
e	0.10, 0.05, 0.02 (± 25 , ± 13 , $\pm 5\%$ a year)
k	0.02 a day (half-life 35 days); 0.01 to 0.10
m, s^2, d	0, 1, 0: the threshold sits at the mean of x
h	1, 2, 5, 10, 21 days between decisions
data	15 years of training, 5 of test, daily



What is measured, over 40 to 100 datasets per cell: where the first threshold lands (target $d = 0$, x has standard deviation 1); the annualized test Sharpe ratio of the tree against the *ideal rule* (the optimizer fed $\mathbb{E}[r | x]$: the best any method can do with x) and the *rule at d* ; the share of the ideal gain captured.

One fitted tree, read on its planted market (dataset 42, weekly, edge 0.10)



What is drawn (x on the decision day, horizontally).

- blue curve (top)**: the true expected 5-day return given x , from the planted model; zero at $x = -0.01$, the planted $d = 0$.
- grey dots**: realized 5-day returns of the training decisions, mean by bin of x (± 1 s.e.).
- orange steps (top)**: the frozen tree's score, one level per leaf: -0.35% for $x \leq 0.12$, $+0.55\%$ up to 1.60 , -0.15% above.
- bottom**: the asset weight the optimizer chooses from a 50/50 portfolio when fed the blue curve (**ideal rule**) or the tree's score (**tree**).

Reading.

- Root threshold 0.12 for a planted 0 . The scores need not match the curve: both sit outside the 0.1% window, so the position is 0 left of 0.12 and 1 right of it, as the ideal rule's, up to 1.60 .
- Above 1.60 the base pipeline kept a leaf with a negative score: cash where the ideal rule stays invested. A few dozen decisions confirmed by a handful of checking rows; the v1 hurdle removes such leaves.
- The ideal rule is a step too: at $\lambda = 1$ the optimizer answers 0 or 1 .

Result 1: the threshold is recovered at market sizes (E2, base pipeline)

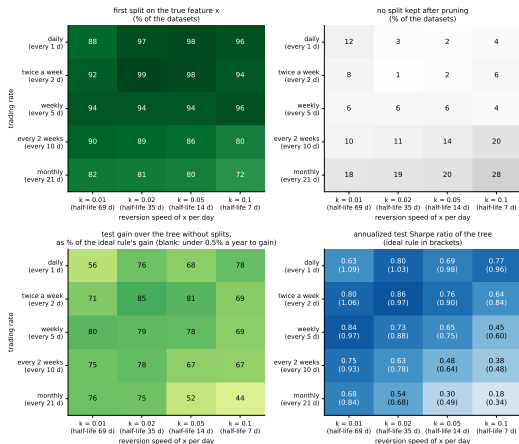
edge	h	split on x (%)	threshold within 0.25	annual. test Sharpe		gain
				tree	ideal	
0.10	1	97	95%	0.80	1.03	76%
	5	94	81%	0.73	0.88	79%
	21	81	60%	0.54	0.68	75%
0.05	1	82	60%	0.12	0.49	25%
	5	78	53%	0.24	0.43	51%
	21	62	34%	0.17	0.35	43%
0.02	5	54	26%	-0.03	0.15	-3%

How to read it. One row = 100 datasets (50 at edge 0.02), each a fresh 15-year history with the threshold planted at $x = 0$. h : days between decisions. *split on x* : share of the datasets whose tree keeps a split at all (with one feature, the alternative is no split). *within 0.25*: share whose first threshold lies within 0.25 standard deviations of x from the planted 0. *Sharpe*: annualized test Sharpe ratio, mean over the datasets (s.e. about 0.04); *ideal* = the optimizer fed the true expected return given x . *gain*: the tree's test return above the tree without splits, as a share of the ideal rule's.

What the table says.

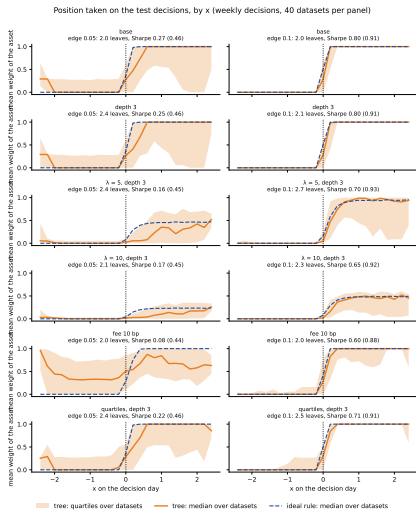
- **With a drift of $\pm 25\%$ a year the threshold is found at every rate.** Almost every tree splits, most thresholds land within 0.25 of the planted 0, and the tree earns 75 to 79% of the ideal rule's gain out of sample.
- **With $\pm 13\%$ it degrades gracefully.** Fewer trees split, the threshold is less precise, and the tree captures 25 to 51% of a gain that is itself small (ideal Sharpe 0.35 to 0.49).
- **With $\pm 5\%$ there is nothing to find in 15 years.** Half of the trees keep no split, the other half split at random ; the tree's Sharpe is 0 and the ideal rule's nearly so (0.15). Finding nothing is the right answer.
- **Rate:** daily to weekly are equivalent; monthly loses precision and Sharpe, because a decision is held while the regime changes.

Result 2: the trading rate against the speed of the feature (edge 0.10)



- Daily to weekly decisions are equivalent: deciding more often than the feature moves adds fees, not information.
- Monthly decisions lose precision and profit, the more so when the feature is fast: with a half-life of 7 days the tree's Sharpe is 0.77 daily and 0.18 monthly (ideal rule 0.96 and 0.34); the regime flips inside the holding period.
- With a slow feature (half-life 69 days) the daily tree under-performs (0.63 against 1.09): 3,780 decisions on a signal that hardly moves make the tree over-trade its own noise.
- Hence **weekly** in v1, with every 2 days as the alternative.

Risk aversion, fees and depth (E3, 1,920 fits): why v1 keeps $\lambda = 1$



Mean position of the asset by bin of x on the test decisions, weekly, edge 0.10, against the ideal rule on the same information.

- $\lambda = 5$ and 10 make the positions gradual, and the tree learns them, at a cost of 0.1 to 0.15 of Sharpe (0.70 and 0.65 against 0.80 weekly).
- A 10 bp fee keeps positions at 0/1 and starves the fast rates: 0.53 against 0.88 every 2 days, turnover halved.
- Values' quartiles as the only candidates lose at the fast rates (0.64 against 0.88 every 2 days).

Decision for v1: $\lambda = 1$, 3 bp, depth at most 3. Sizing is a later question (at $\lambda = 10$ the tree does size as the ideal rule does).

Split-search options: quantile candidates, refinement, soft splits

At a leaf with rows I (N rows) a child must keep $m_* = \max(m, \lceil \varphi N \rceil)$ rows ($m = 20$, $\varphi = 10\%$ in v1). The **plain** candidates of a feature are the midpoints between consecutive distinct values, $\theta_{(1)} < \dots < \theta_{(K)}$, whose two sides both keep m_* rows. Each candidate is **screened** cheaply, each child getting its clipped mean return as score:

$$Q(\theta) = \sum_{t \in I} \text{Reg}_t(c_{\text{side}(t)}(\theta)), \quad c_{\text{left}}(\theta) = \text{clip}(\text{mean}\{r_t : x_t \leq \theta\}, \pm b), \text{ likewise on the right.}$$

Quantile candidates + refinement (on in v1)

- *Deciles by rank.* Instead of all K candidates, keep the nine at ranks $\text{round}(q(K-1))$, $q = 0.1, \dots, 0.9$: evenly spread over the admissible range, never on its edge. The full screen costs $K \approx N - 2m_*$ solves, this costs 9.
- *Refinement.* Let θ_o be the best decile. Screen every plain candidate strictly between its two neighbouring deciles; let $\theta_\bullet$ be the best of them. It replaces θ_o only if the improvement is clear,

$$Q(\theta_o) - Q(\theta_\bullet) > k \cdot \text{se}, \quad \text{se} = \sqrt{|I|} \text{std}_t(\delta_t),$$

δ_t the per-row difference of the two regrets, $k = 1$ in v1.
The 3 best survivors get the full fit of their children.

Soft split (off in v1: $\kappa = 0$)

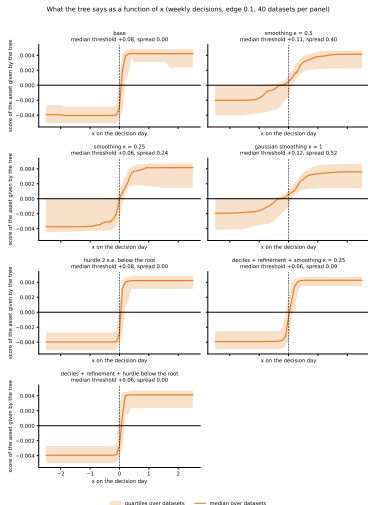
- Keep *all* candidates $\theta_1 < \dots < \theta_K$ of the feature, with Boltzmann weights from their screened regrets:

$$z_k = \frac{Q(\theta_k) - \min_l Q(\theta_l)}{\text{se}_k}, \quad \pi_k = \frac{e^{-z_k/\kappa}}{\sum_l e^{-z_l/\kappa}}$$

(Gaussian variant $e^{-z^2/2\kappa^2}$): thresholds the data cannot tell from the best share the weight, clearly worse ones get none.

- A row with value x goes left with weight $\mu(x) = \sum_{k: \theta_k \geq x} \pi_k$, right with $1 - \mu(x)$; weights multiply down the tree ($\omega_{t\ell}$), the leaf fit minimises $\sum_t \omega_{t\ell} \text{Reg}_t(c)$, and a row's score is the mixture $\hat{c}(x_t) = \sum_\ell \omega_{t\ell} \hat{c}_\ell$: a ramp in x instead of a jump. As $\kappa \rightarrow 0$ the plain split is recovered.

Options tried, 1: candidate thresholds and soft splits (E4, E8; 3,840 fits)



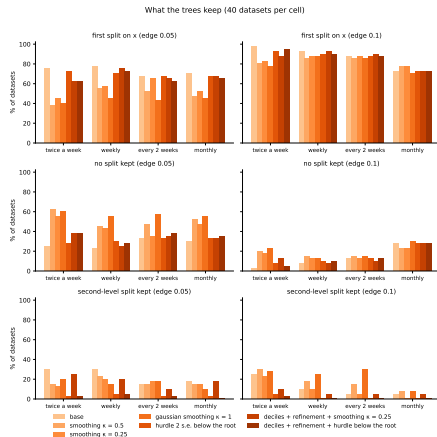
The tree's score as a function of x , weekly, edge 0.10 (median and quartiles over 40 datasets).

- **Deciles by rank + local refinement** (in v1): screen the deciles of the admissible thresholds, then the neighbourhood of the best one. Sharpe 0.82 against 0.80 for the base weekly; thresholds steadier at the slow rates (within 0.25 of d monthly: 57% against 45%); ten times fewer candidates. It buys stability, not Sharpe.
- **Soft splits** (Boltzmann weights over the candidates; off in v1): at $\kappa = 1$ the transition is 0.65 wide and the averaged score falls in the optimizer's no-trade band, so the tree stops trading (0.53 against 0.80 weekly, 0.35 against 0.87 every 2 days). At $\kappa = 0.25$ the spread falls to 0.2 and the tree matches the base from weekly on, but still loses 0.26 every 2 days. On top of deciles and refinement it gives the narrowest transition (0.1) and the best weekly numbers (0.82, 85% of the gain), within noise of the base.
- Verdict: at $\lambda = 1$ smoothing the score does not size the position; the option stays in the code for $\lambda \geq 5$.

Options tried, 2: a calibrated pruning hurdle (E5, E8; 3,840 fits)

The pruning gain of a split is $\bar{g}$ over the checking rows; the hurdle asks $\bar{g} > k_p \text{se}(\bar{g})$.

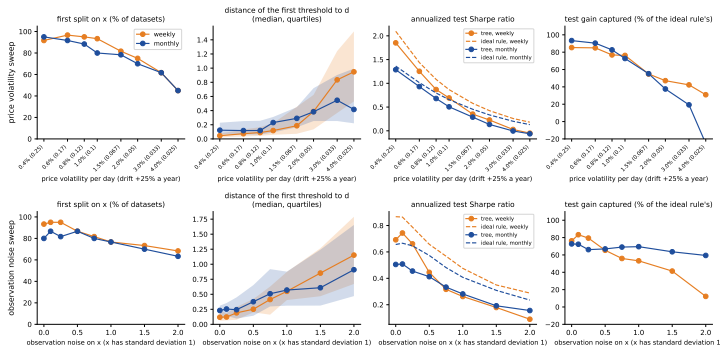
- **Global hurdle** ($k_p = 1$ or 2 at every node): removes the spurious second-level leaves but loses true root splits, 60% found at 2 s.e. against 98% every 2 days, Sharpe 0.47 against 0.87. One quarter of the decisions (189 weekly) has not the power to confirm a real split at 2 s.e.
- **Hurdle 2 s.e. below the root** (in v1): second-level splits fall from 25/10/5/5% of the datasets to 5/0/0/0% (2-day to monthly), the root is untouched, the Sharpe ratio is unchanged from weekly on (0.77, 0.68, 0.57 against 0.80, 0.69, 0.57), at edge 0.05 too.
- Its cost: -0.13 every 2 days, where with 1,400 fitting decisions some second-level splits are real.



Share of datasets with a first split on x , no split, and a second-level split, by option and rate (edge 0.10 left, 0.05 right).

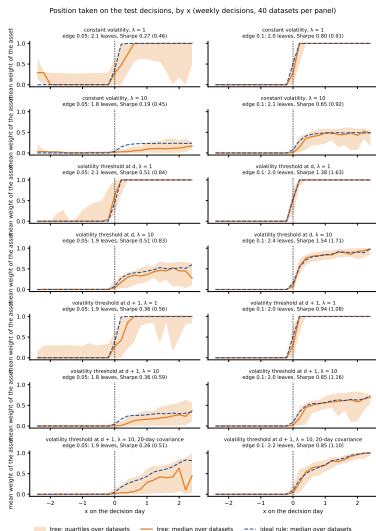
What can be found at all: resolution against volatility and noise (E6, 1,920 fits)

Resolution of the threshold (60 datasets per point; in brackets: the edge, drift over volatility per day)



- Drift fixed at $\pm 25\%$ a year, volatility swept: the threshold error is about $0.012/e$ standard deviations of x until the edge falls below 0.05, where the split becomes a guess (median $|\theta|$: 0.05 at $e = 0.25$, 0.12 at 0.10, 0.39 at 0.05, 0.95 at 0.025).
- White noise of τ standard deviations on the observed feature acts like dividing the edge by $1 + \tau^2$: up to 0.25 costs nothing, 0.5 doubles the error, 1 halves the Sharpe of the ideal rule itself.
- An edge below a few basis points a day will not be learnt from 15 years of data by any method: test a real feature with this protocol before trading it.

A second market: the volatility follows a threshold too (E7, 2,240 fits)



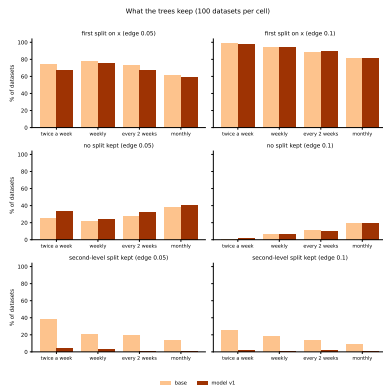
$\sigma_t = 0.5\%$ above d_σ , 1% below; $d_\sigma = d$ (stress = bad regime) or $d_\sigma = d + 1$ (a calm part inside the good regime, where the right position is *smaller*, not of the other sign); $\lambda = 1$ and 10.

- With $d_\sigma = d$ the drift threshold is found better: within 0.25 of d in 95% of the weekly datasets (84% with constant volatility); every Sharpe ratio nearly doubles; 83–86% of the ideal gain kept. Clean returns in the good regime pin the threshold.
- The volatility boundary at $d + 1$ is not learnt: 8–12% of the trees split near $x = 1$; calm-regime position 0.52 for the tree, 0.58 for the ideal rule on the same information, 0.96 for the clairvoyant rule.
- Structural: the regret uses the same lagged 180-day covariance as the optimizer, which averages over regimes, so a boundary that exists only in the true variance has nothing to gain in the loss. *The tree cannot learn what its own utility does not reward.*
- A 20-day window follows the regime (calm position 0.69–0.75) but its noise costs more than the tracking gains. For v2: a regime-aware variance, not a deeper tree.

Model v1 on 100 datasets (E9, 800 fits), paired with the base of Result 1

edge	h	leaves		2nd level	within 0.25	Sharpe		v1 — base (2 s.e.)
		base	v1	base → v1	base → v1	base	v1	
0.10	2	2.24	2.00	25 → 2%	92 → 89%	0.86	0.78	-0.08 (0.07)
	5	2.14	1.94	18 → 0%	81 → 82%	0.73	0.73	+0.00 (0.04)
	10	2.03	1.92	14 → 2%	70 → 72%	0.63	0.64	+0.01 (0.05)
	21	1.90	1.81	9 → 0%	60 → 64%	0.54	0.53	-0.01 (0.03)
0.05	2	2.18	1.72	38 → 4%	64 → 57%	0.21	0.18	-0.03 (0.06)
	5	2.01	1.80	21 → 3%	53 → 51%	0.24	0.22	-0.03 (0.05)
	10	1.93	1.69	19 → 1%	47 → 49%	0.17	0.20	+0.03 (0.04)
	21	1.76	1.59	13 → 0%	34 → 42%	0.17	0.14	-0.03 (0.03)

- From weekly on, v1 is the base within noise: second-level leaves gone, 1.9 leaves instead of 2.1, thresholds as well centred.
- Every 2 days it costs 0.08 ± 0.07 at edge 0.10: the hurdle removes second-level splits that are real there; the 2-day alternative needs a softer hurdle (not run).
- The depth budget of 3 is never used here (at most 1% of the trees reach the third level).
- Verdict: **v1 is the base with a cleaner structure, not a higher Sharpe ratio.**



Same 100 datasets (seeds 0–99). “2nd level”: datasets keeping a second-level split; “within 0.25”: first threshold within 0.25 of d ; last column: mean paired difference of annualized test Sharpe, twice its s.e. in brackets. In half of the datasets the two trees give the same Sharpe ratio.

A fair assessment

What holds

- The mechanism works at market sizes: a $\pm 25\%$ regime on a monthly feature is recovered at every rate and 75–85% of the attainable gain is earned out of sample; at $\pm 13\%$ it degrades gracefully; at $\pm 5\%$ it correctly finds nothing.
- A regime that also drives the volatility is easier, not harder, when the calm regime is the profitable one.
- The v1 additions (deciles + refinement, hurdle below the root) make the trees smaller and the thresholds steadier at no measurable cost from weekly on (100 paired datasets).
- Everything is reproducible: results files, figures, dated history, 64 tests.

What is weak or untested

- **No option beat the base beyond noise** (s.e. about 0.04 on 40 datasets), nor does v1 on 100. v1 buys stability, not Sharpe, and loses 0.08 every 2 days.
- **The growth gate on the replayed Sharpe is in-sample** and mixes a risk preference into the structure search; kept as designed, it is the first thing v2 tests (gate off, or the replayed regret instead).
- **The pruning judge is one 25% block**: too little power for a global hurdle; a split no checking row reaches cannot be confirmed.
- **One clean feature**. Decoys were only tested at an unrealistic edge (0.95, E1); several features and interactions are untested at market sizes.
- **Synthetic data only**. No real-data run, no neural benchmark.
- **Approximate leaf search**; bang-bang positions at $\lambda = 1$; volatility regimes invisible to the loss with a lagged covariance.

Next steps (towards v2)

1. **The growth gate.** Run v1 with the Sharpe gate off, and with the path-replayed regret as the confirmation instead; same 100 datasets, paired.
2. **A regime-aware variance** for the optimizer and the regret (a variance driven by the feature, or an exponentially weighted estimate), then check that the second split appears at d_σ in the regime market.
3. **Decoys and interactions at market sizes:** two features, four regimes, plus unrelated features; does the tree still ignore the decoys and find the interaction with a depth-3 budget?
4. **Feature denoising:** an exponentially weighted feature with the exact Gaussian-conditioned ideal rule as the ceiling (E6 says noise above 0.25 std matters).
5. **Position sizing:** $\lambda \geq 5$ with soft splits, where smoothing the score does size the position.
6. **Real data** (the template `run_real_data.py`), with the planted-threshold protocol as the pre-test of every candidate feature; then the neural benchmark of the motivations note.

Appendix: the experiments behind the slides ([model_history.md](#))

id	date	market, pipeline	fits	headline
E1	18 Sep	M1 (edge 0.95, 3 decoys), P1	800	x found first in 86–100% of the datasets, never a decoy; drift far too large to be realistic
E2	22 Sep	M2, P2 (base)	2,000	threshold recovered at every rate at $\pm 25\%/yr$, three times in four at $\pm 13\%$, not at $\pm 5\%$; monthly loses precision
E3	22 Sep	M2, P2	1,920	depth alone changes nothing; $\lambda = 5-10$ gives gradual positions at -0.1 Sharpe; 10 bp fees starve the fast rates
E4	23 Sep	M2, P3b	1,600	deciles + refinement $\approx$ base with steadier thresholds; smoothing $\kappa = 1$ too wide, inside the no-trade band
E5	23 Sep	M2, P3b	1,600	a global pruning hurdle removes spurious leaves but loses true root splits
E6	23 Sep	M2' (noise), P3b	1,920	threshold error $\approx 0.012/e$ until $e \approx 0.05$; noise τ acts like $edge/(1 + \tau^2)$
E7	23 Sep	M2' (regime volatility), P3b	2,240	drift threshold found better with $d_\sigma = d$; the volatility boundary is not learnt (lagged covariance)
E8	23 Sep	M2, P4	2,240	$\kappa = 0.25$ cures the no-trade problem, still -0.26 at 2 days; hurdle below the root removes second-level leaves at no cost from weekly on
E9	24 Sep	M2, P4 in configuration v1	800	v1 = base within noise from weekly on (0.00 ± 0.04 weekly at edge 0.10), second-level leaves gone; -0.08 ± 0.07 every 2 days (slide 21)

Pipelines: P1 first tree (18 Sep), P2 all-cash Sharpe = 0 (22 Sep), P3/P3b split-search options and rank quantiles (23 Sep), P4 Gaussian kernel and depth-restricted hurdle (23 Sep evening). Markets: M1 the first world, M2 the daily market with the threshold at the mean, M2' with observation noise and regime volatility as options. Reproduce: `python planted_threshold_experiments.py run | run-settings | run-enhancements | run-pruning | run-resolution | run-regime | run-options | run-v1 | plot from code/`.